Wherever you see **red characters**, these need to be replaced by your information. This includes the **< >** symbols!

Looking at Variables / Columns Quickly

```
table(<NAME OF DATASET>$<NAME OF VARIABLE>)
summary(<NAME OF DATASET>$<NAME OF VARIABLE>)
```

The $ operator pulls a variable or column from a dataset.

Sorting / Arranging Rows in a Table

```
<NAME OF DATASET> |>
    arrange(<NAME OF VARIABLE>)
```

Sorts the rows of the dataset from smallest to largest values of the given variable.

```
<NAME OF DATASET> |>
    arrange(desc(<NAME OF VARIABLE>))
```

Sorts the rows of the dataset from largest to smallest values of the given variable.

```
<NAME OF DATASET> |>
    arrange(<NAME OF VARIABLE 1>,
            <NAME OF VARIABLE 2>,
             …)
```

Sorts the rows of the dataset from smallest to largest values of variable 1 and then variable 2.

Nicely Formatting a Table

```
<TABLE CODE…> |>
  kable(digits = <# significant digits>,
        col.names = c("<YOUR NICE COL NAME 1>",
                      "<YOUR NICE COL NAME 2>",
                      "<YOUR NICE COL NAME 3>",
                      … ))
```

Controls the visual of a table. The number of "nice" column names given for `col.names` **must** be equal to the number of columns in the table. Requires `kableExtra` package.

Boxplots with Individual Data Points

```
ggplot(data = <NAME OF DATASET>,
       aes(x = <NAME OF QUANT VAR>, y = <NAME OF CATEGORICAL VAR>)) +
       geom_boxplot() +
       geom_jitter(color = "blue",
                   width = 0, height = .2)
```

Adds blue ndividual data points on top of boxplots with some random noise added so the points don't overlap.

Wherever you see **red characters**, these need to be replaced by your information. This includes the **< >** symbols!

Pooled Two-Sample T-Test

```
t.test(<NAME OF RV> ~ <NAME OF EV>,
        var.equal = TRUE,
        alternative = “two.sided”,
        data = <NAME OF DATASET>)
```

Runs a **pooled** two sample t-test with a two-sided alternative hypothesis. For one-sided tests use `alternative = “greater”` or `alternative = “less”`.

ANOVA F-Test

```
mymod <- lm(<NAME OF RV> ~ <NAME OF EV>,
                data = <NAME OF DATASET>)
anova(mymod)
```

Runs a theory-based ANOVA F-test. Output is the ANOVA table.
Note that you can call **`mymod`** anything you want but then you must use the same name of that model that you created in the `anova()` function.

Post-Hoc Pairwise Comparisons

```
aov(<NAME OF RV> ~ <NAME OF EV>,
    data = <NAME OF DATASET>) |>
    emmeans(specs = "<NAME OF EV>") |>
    pairs() |>
    confint(adjust = "none")
```

Provides all pair-wise comparison 95% confidence intervals.

```
aov(<NAME OF RV> ~ <NAME OF EV>,
    data = <NAME OF DATASET>) |>
    emmeans(specs = "<NAME OF EV>") |>
    pairs(adjust = "none")
```

Provides all pair-wise comparison pooled t-test p-values.

```
aov(<NAME OF RV> ~ <NAME OF EV>,
    data = <NAME OF DATASET>) |>
    emmeans(specs = "<NAME OF EV>") |>
    pairs() |>
    confint(adjust = "none")|>
    data_frame() |>
    arrange(desc(abs(estimate)))
```

Provides all pair-wise comparison 95% confidence intervals, organized by largest to smallest absolute difference in means between groups.

Wherever you see **red characters**, these need to be replaced by your information. This includes the **< >** symbols!

Calculating a p-value with a F-statistic

```
pf(q = <VALUE OF OBSERVED F-STATISTIC>,
    df1 = <VALUE OF MODEL DF>, df2 = <VALUE OF ERROR DF>,
    lower.tail = FALSE)
```

Calculates the area under the F(df1, df2) distribution that is greater than or equal to a given F-statistic.

Calculating the t* critical value for a Confidence Interval

```
qt(p = <VALUE OF DESIRED PERCENTILE (SEE NOTE)>,
    df = <VALUE OF DF>)
```

Calculates the t^*_{df} for a confidence interval for a mean or difference in means given the degrees of freedom and confidence level. To calculate the correct value for a $(1-\alpha)$% CI, the desired percentile is $1-\frac{\alpha}{2}$. For example, for a 95% confidence interval, set `p = 0.975`.